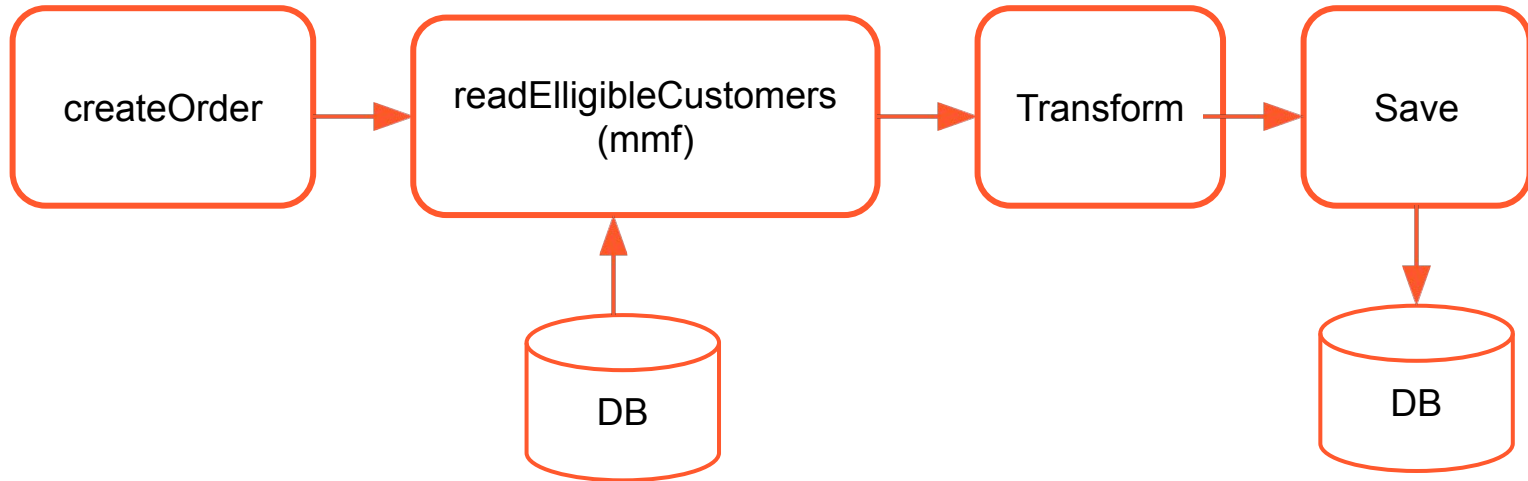


Scaling Data Processing in Kotlin without Melting Your Memory.

Use case

Use Case: Process Eligible Customers

```
val mmfs = listOf("Amundi", "DWS", "Blackrock")
```



ProcessEligibleCustomers

MMF: Amundi

Match: 10% of total data

Read time: 30mins



ProcessEligibleCustomers

MMF: DWS

Match: 40% of total data

Read time: 2h



ProcessEligibleCustomers

MMF: Blackrock

Match: 50% of total data

Read time: 3h

Eager List Allocations vs. "Millions of Short-Lived Objects"

Eager List Allocations vs. "Millions of Short-Lived Objects"

```
val mmfs = listOf("Amundi", "DWS", "Blackrock")  
mmfs.forEach(){ mmf ->  
    val customers = db.readCustomersByMmfAsList(mmf) // 100k customers  
        .filter { it.isEligible }  
        .map { processCustomer(it) }  
}
```

Total: 200k "Short-Lived Objects"

JVM allocated memory for 200k

These intermediate lists become garbage almost immediately after creation.



Loads all matched data at once

Improve application memory with with Sequences

How: Going Lazy with Sequences

```
val mmfs = listOf("Amundi", "DWS", "Blackrock")  
mmfs.forEach(){ mmf ->  
    val customers = db.readCustomersByMmfAsSeq(mmf) // 1 item x time  
        .filter { it.isEligible }  
        .map { processCustomer(it) }  
        .forEach { chunk -> } // Terminal operation  
}
```

Because no intermediate collections are created, you **drastically reduce** the number of temporary objects hitting the JVM Heap.



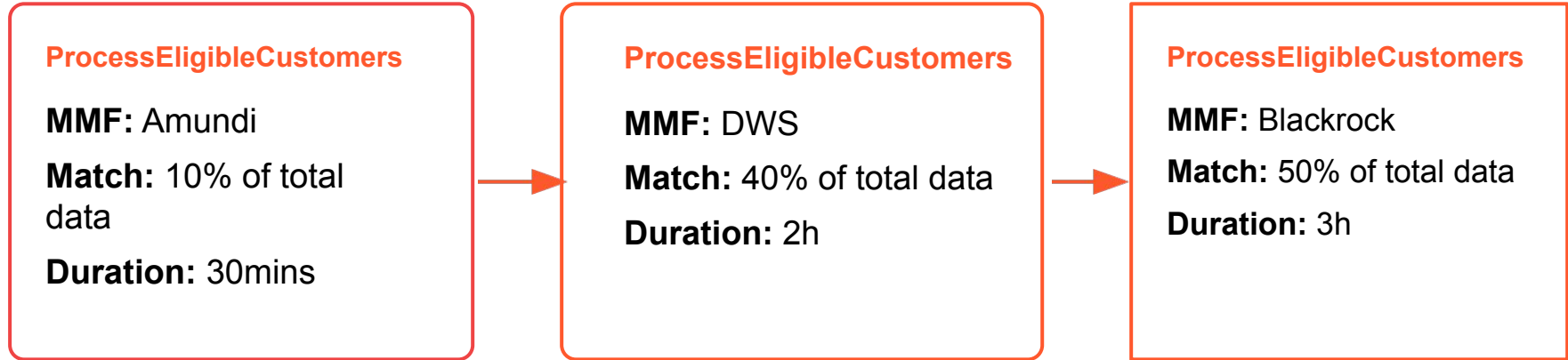
Single-Threaded Execution

While memory is saved, it's still running **sequentially** on a single thread.



Latency Sensitivity

It's safe, but not necessarily fast if individual steps like **DB or API calls** are slow.



Total processing time = 5h30

Lazy fetch from DB with Jooq

List

fetch

Results in a Java List (eager)

Cursor

fetchLazy

Internal reference to

`ResultSet`

Requires `fetchSize`

Stream

Uses Java Stream API

Holds internal reference to

`ResultSet`

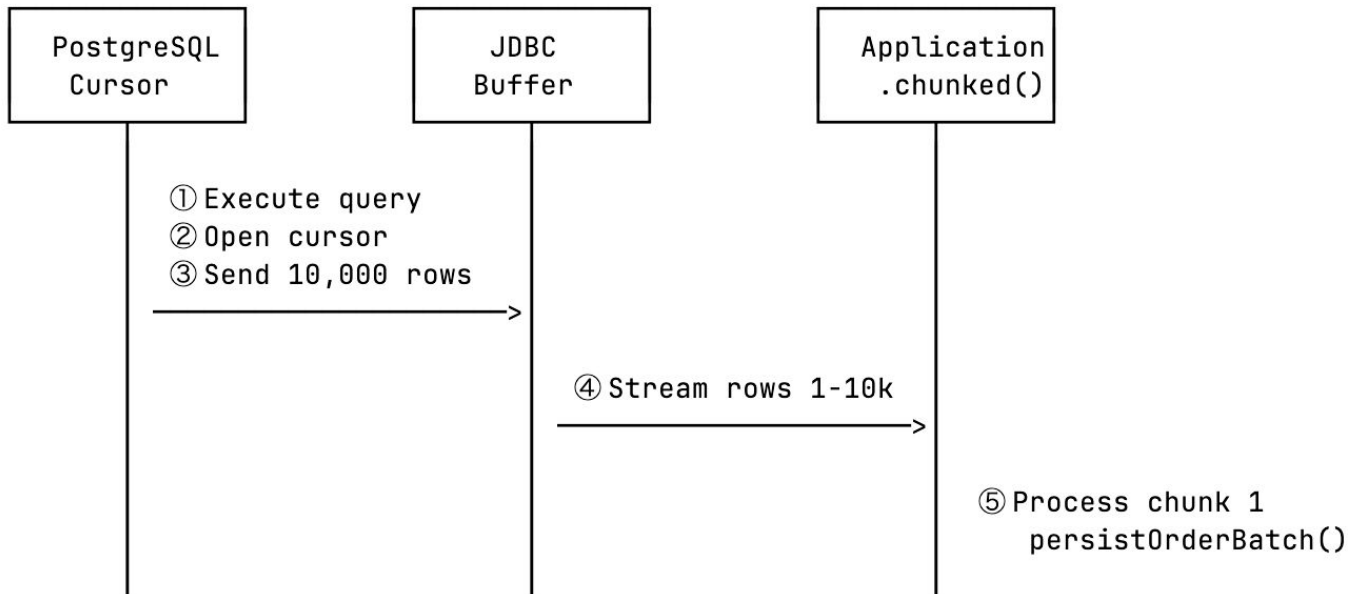
```
query  
  .fetch()
```

```
query  
  .fetchSize(Int)  
  .fetchLazy()
```

```
query  
  .fetchSize(Int)  
  .fetchStream()  
  .asSequence()
```

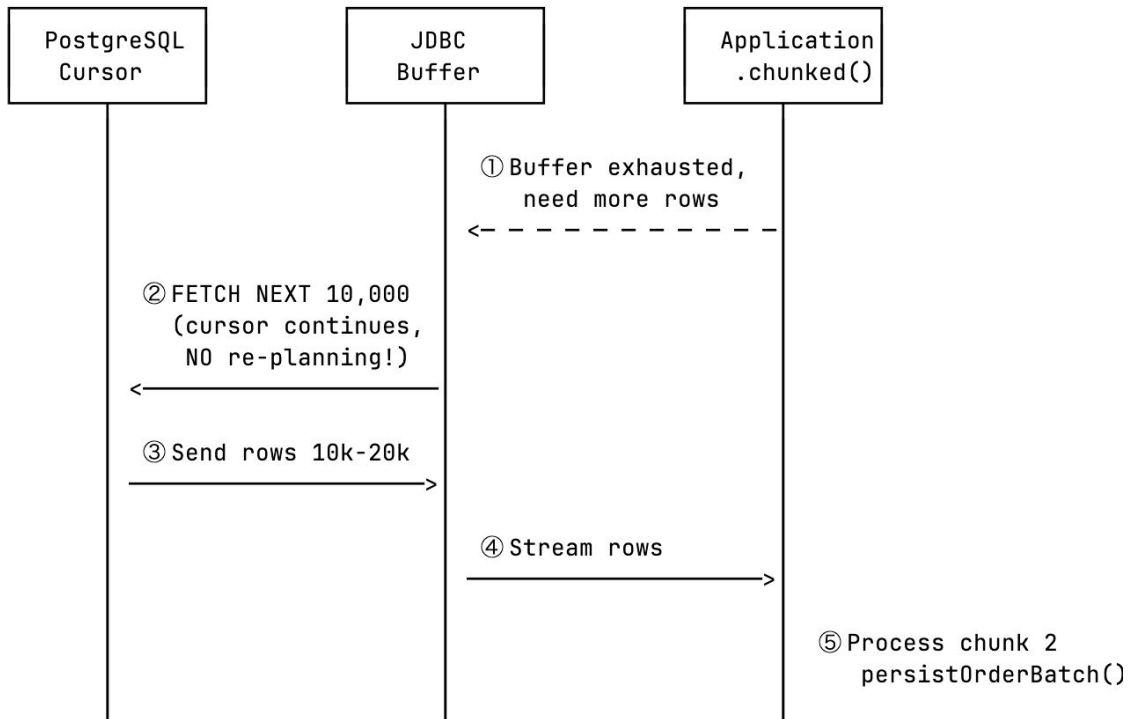
How stream fetching works

FIRST FETCH (rows 1 - 10,000)



How stream fetching works

SECOND FETCH (rows 10,001 - 20,000)



Phase 1: Query Plan Execution (ONCE)

```
SELECT * FROM customers WHERE eligible = true ORDER BY id
```

Matches 20,000 rows

Returns first 10,000

Phase 2: Server-Side Cursor (ACTIVE)

Holds the result set and maintains the read state until the stream is fully consumed.

Read Position:



^ Current read position

Row Index: [1][2][3] ... [9999][10000] [10001] ... [20000]

TCP/Network ▼

JDBC DRIVER

Fetch Buffer (fetchSize = 10,000)

Holds up to 10,000 rows in memory

Current: [1] [2] [3] ... [9998] [9999] [10000]

ResultSet.next() ▼

JOOQ STREAM / KOTLIN SEQUENCE

`.fetchStream().asSequence()` → Lazy iterator over `ResultSet`

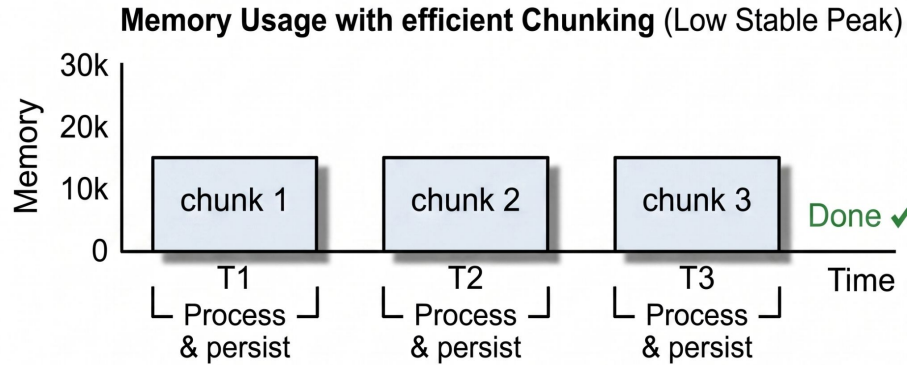
`.chunked(chunkSize = 10,000)` ▼

APPLICATION PROCESSING

Chunk 1: [1..10,000] → `persistOrderBatch()` → ✓ Done

Chunk 2: [10,001..20,000] → (waiting...)

Chunk 3: [20,001..25,000] → (not yet)



Unleashing Speed with Flow



Asynchronous: Lightweight concurrency in a sequential style.



Non-blocking: Handle time-consuming tasks without freezing the underlying thread.



Resource Efficient: Significantly more lightweight than traditional OS threads.



Structured Concurrency: Manage lifecycles to avoid leaks and inconsistent states.



Advanced abstraction of coroutines



Non-blocking Operations



Cold Streams

Sequential flow

```
flowOf("Amundi", "DWS", "Blackrock")  
.collect { mmf -> processEligibleCustomers (mmf) }
```

ProcessEligibleCustomers

MMF: Amundi

Match: 10% of total
data

Duration: 30mins

ProcessEligibleCustomers

MMF: DWS

Match: 40% of total data

Duration: 2h

ProcessEligibleCustomers

MMF: Blackrock

Match: 50% of total data

Duration: 3h

Total processing time = 5h30

Asynchronous Concurrency (flatMapMerge)

```
val mmfs = listOf("Amundi", "DWS", "Blackrock")

mmfs.asFlow()
  .flatMapMerge(concurrency = 3) { mmf ->
    db.readLargeEligibleDataset(mmf).asFlow() // Seq. to flow
  }
  .map { customer -> processLightweightAppLogic(customer) }
  .collect { result -> db.writeResult(result) }
```



The application opens **three separate database connections/cursors at the same time**.

Asynchronous Concurrency (flatMapMerge)

```
Thread-1: [====BLOCKING DB READ====] [emit]→[====BLOCKING DB READ====] [emit]  
→...  
Thread-2: [====BLOCKING DB READ====] [emit]→[====BLOCKING DB READ====] [emit]  
→...  
Thread-3: [====BLOCKING DB READ====] [emit]→[====BLOCKING DB READ====] [emit]  
→...
```

- Sequence iteration is **synchronous and blocking**
- `asFlow()` wraps it, but each `next()` call blocks the thread while fetching
- With concurrency = 3, you block 3 threads from your dispatcher
- Backpressure still works, but the thread sits idle waiting on I/O

Concurrent reads from DB

```
fun readLargeEligibleDataset(mmf: String): Flow<Customer> = flow {  
  
    val lazyQuery = query  
        .fetchLazy()  
        .fetchSize(10k)  
  
    try {  
        // Stream rows continuously out of the open cursor  
        for (row in lazyQuery) { // Block I/O Threads only  
            emit(row.toCustomerDomainModel()) // Push single customer down the stream  
        }  
    } finally {  
        lazyQuery.close() // Always close the cursor when the flow completes  
    }  
  
}.flowOn(Dispatchers.IO) // Blocking isolated to IO pool
```

Concurrent processing pipeline

```
val mmfs = listOf("Amundi", "DWS", "Blackrock")

mmfs.asFlow()

// 1. flatMapMerge opens 3 concurrent DB streams at the same time
.flatMapMerge(concurrency = 3) { mmf ->
  db.readLargeEligibleDataset(mmf) // Returns Flow<Customer>
}

// 2. Lightweight processing happens on individual customers as they stream in
.map { customer ->
  processLightweightAppLogic(customer)
}

// 3. Write results back to the database as they finish processing
.collect { result ->
  db.writeResult(result)
}
```

Concurrent processing pipeline

```
IO Thread:  [==DB READ==]           [==DB READ==]           [==DB READ==]
              ↓ emit                  ↓ emit                  ↓ emit
Compute Thread:  [map][collect]      [map][collect]      ...
```

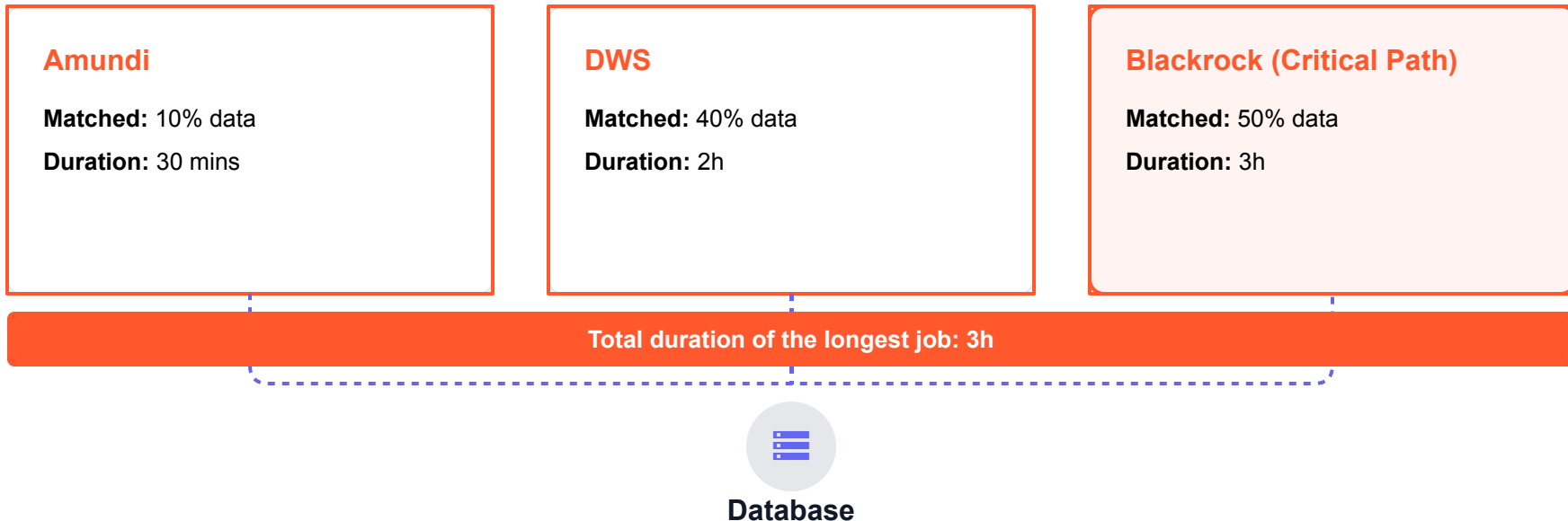
- The blocking `DB` work is isolated to `Dispatchers.IO`
- Downstream `map/collect` can run on a different dispatcher
- The **Flow** structure gives you explicit control over threading context
- Cancellation propagates properly—if `collect` stops, the cursor closes

Fetch sequence vs fetch flow

Aspect	Sequence → asFlow()	Native Flow
Thread blocking	Blocks caller's thread during iteration	Can isolate blocking with <code>flowOn</code>
Dispatcher control	Harder to separate concerns	<code>flowOn</code> moves upstream work to specific dispatcher
Cancellation	Works, but cleanup less explicit	<code>finally</code> block in flow builder is idiomatic
Composition	Sequence logic is synchronous	Can mix with delay, other suspend funs
Mental model	"Pull one, emit one, block"	"Emit when ready, suspend between"

Asynchronous Concurrency (flatMapMerge)

Total execution time is determined by the slowest parallel task



Database Connection Pool Starvation

```
flatMapMerge(concurrency = 20)
```



Pool Limit = 10



Other parts of the service will be **waiting** for connections to free up, causing system-wide latency or failures.

When to use which (non exhaustive)

Ideas of when to use which (non exhaustive)

Scenario	Best Choice	Why / Caveat
Small data, need all results	List	Eager, simplest, and fastest. No lazy iteration overhead.
Large data, need subset (.take())	Sequence	Lazily evaluates one item at a time. Short-circuits perfectly.
Large data, CPU-bound batching	Sequence + chunked	Keeps memory flat while processing chunks of heavy computations.
Large data, Async/I/O batching	Flow + chunked	Use if the batch needs to be sent to a DB or API asynchronously without blocking the thread.

Repo



Slides





THANK
YOU! 